

apter]ListingEnvlolЛистинг

Федеральное государственное автономное образовательное учреждение
высшего образования «Длинное название образовательного учреждения
«АББРЕВИАТУРА»



На правах рукописи

Фамилия Имя Отчество

**Длинное название диссертационной работы, состоящее
из достаточно большого количества слов, совсем длинное
длинное длинное длинное название, из которого простому
обывателю знакомы, в лучшем случае, лишь отдельные
слова**

Специальность XX.XX.XX —

«Технология обработки, хранения и переработки злаковых, бобовых культур,
крупяных продуктов, плодоовощной продукции и виноградарства»

Диссертация на соискание учёной степени
кандидата физико-математических наук

Научный руководитель:
уч. степень, уч. звание
Фамилия Имя Отчество

Город — 2025

Оглавление

	Стр.
Введение	6
Глава 1. Картографирование окружающего пространства робота на основе комплексирования данных разной природы	10
1.1 Модели окружающего пространства робота	10
1.2 Обзор используемых в колёсной робототехнике датчиков и их ограничения	10
1.3 Классификация методов комплексирования данных	10
1.4 Требования к алгоритму картографирования окружающего пространства робота	10
1.4.1 Метрика оценки качества карты проходимости пространства $PFC - MSE$	10
1.5 Заключение к главе 1	10
Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения робота	11
2.1 Алгоритм проекции визуальных детекций на плоскость движения робота на основе модели L-shape	11
2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-shape	11
2.3 Алгоритм проекции визуальных детекций на плоскость движения робота на основе комплексирования данных с данными лазерных дальномеров	11
2.4 Исследование точности алгоритма проекции визуальных детекций на основе комплексирования данных с данными лазерных дальномеров	11
2.5 Алгоритм проекции сегментации проезжей части на изображении на плоскость движения ТС	11

2.6	Исследование точности алгоритма проекции визуальных детекций на основе комплексирования данных с данными лазерных дальномеров	11
2.7	Заключение к главе 2	11

Глава 3. Программный комплекс построения карты проходимости пространства на основе

	комплексирования разнородных данных	12
3.1	Назначение и функциональные возможности	12
3.2	Структура программного комплекса	12
3.2.1	Библиотека <i>occipancy_map_fusion</i>	12
3.2.2	Программа <i>evo_occipancy_map_fusion</i>	12
3.3	Заключение к главе 3	12

Глава 4. Неблокирующий алгоритм построения карты проходимости пространства на основе

	комплексирования разнородных данных	13
4.1	Introduction	13
4.2	Постановка задачи	18
4.3	Модификации базовой реализации	23
4.3.1	Base OGM	23
4.3.2	OGM with prefetched areas	26
4.3.3	OGM with lockfree cell value update	27
4.3.4	Fully lockfree OGM	31
4.4	Сравнение задержки обновления локальной карты проходимости пространства	34
4.4.1	Сдвиг карты	35
4.4.2	Добавление фрагмента в одном потоке	35
4.4.3	Добавление фрагмента в нескольких потоках	36
4.4.4	Получение карты	37
4.4.5	Стандартный цикл "публикации" карты	40
4.5	Реализация для 2D случая	41
4.6	Выводы	41

4.7	Исследование ограничений алгоритма построения карты проходимости пространства с блокирующими операциями	44
4.8	Модификации алгоритма построения карты проходимости пространства с блокирующими операциями	44
4.9	Исследование вычислительной сложности модификаций алгоритма построения карты проходимости пространства	44
4.10	Заключение к главе 4	44
Заключение		45

Введение

Последние десятилетия характеризуются активным внедрением и распространением беспилотных транспортных средств (ТС) в различных сферах человеческой деятельности. Примерами таких сфер являются логистика, промышленность, сельское хозяйство, горное дело, городская инфраструктура. Для осуществления успешной навигации ТС требуется модель окружающего пространства. В зависимости от выполняемых задач к данным моделям предъявляются различные требования, включая количество параметров, описывающих модель, разрешающую способность, вычислительную сложность построения. Генерация модели производится на основании показаний различных датчиков, которыми оснащается ТС. Расположение, количество и тип используемых датчиков зачастую уникально для каждой модели ТС. Эти характеристики требуют учёта в выбираемой модели окружающего пространства для обеспечения своевременного учёта показаний всех доступных регистрируемых данных и, как следствие, достижения качественного и безопасного решения задач.

Задача построения модели окружающего пространства в робототехнике широко исследована, однако остаётся решённой не в полной степени и сегодня. Исследование этой задачи нашло отражение в работах ... Одной из таких моделей, нашедшей широкое применение в алгоритмах для колёсных роботов, является карта проходимости пространства, представляющая окружающее ТС пространство в виде сетки, каждой клетке, которой сопоставляется величина, характеризующая вероятность нахождения препятствия в соответствующей области. Данная модель, вместе с последующими модификациями была исследована в работах Э. Альберто, С. Труна, Л.М.Г. Гонсалвеса, Л. Хеннинга. Для построения более полной модели, как правило, используется не один датчик, а их набор. Использование нескольких датчиков позволяет, во-первых, уменьшить слепую зону ТС, а, во-вторых, при использовании датчиков разного типа, компенсировать ограничения каждого типа сенсора по-отдельности. Поэтому, важным аспектом при разработке моделей окружающего пространства, является способ комплексирования разнородных данных регистрируемых датчиками для построения полного и непротиворечивого описания окружения. Среди исследований задачи комплексирования данных стоит выделить работы

Б. Халеки, Б.М. Миллера, П.П. Николаева, Ю.В. Визильтера. Однако вопрос применимости алгоритмов комплексирования при увеличении числа и/или типов комплексируемых датчиков не был в достаточной степени исследован к настоящему времени.

Целью данной работы является повышение надежности автономного управления за счёт разработки методов построения моделей окружающего пространства, которые работают на бортовом вычислителе ТС и используют алгоритмы комплексирования данных о проходимости, динамике объектов в сцене, а также слепых зонах ТС.

Для достижения поставленной цели необходимо было решить следующие **задачи**:

1. Исследовать существующие методы построения моделей окружающего пространства ТС
2. Разработать уникальный алгоритм оценки положения объектов в окружающем пространстве ТС
3. Разработать уникальные алгоритмы комплексирования данных в модель окружающего пространства ТС
4. Разработать комплекс технических средств, обеспечивающий построение модели окружающего пространства ТС, учитывающей наличие динамических и статических препятствий в сцене

Научная новизна:

1. Предложены новые алгоритмы проекции визуальных детекций и сегментации проезжей части на изображении на плоскость движения ТС на основе комплексирования данных с изображения и лидарного облака
2. Предложен новый метод построения карты проходимости пространства, позволяющий снизить время задержки между моментом регистрации данных сенсором и моментом обновления карты

Практическая значимость. Разработанные алгоритмы и комплекс технических средств, обеспечивающие построение модели окружающего пространства ТС, реализованные в данной работе, внедрены в автономные беспилотные транспортные средства “EVO.1” и “EVO.3”, разработанные компанией ООО “Эвокарго”. Внедрение разработанных в данной диссертации методов подтверждено соответствующими актами о внедрении. На результаты, полученные в работе были получены свидетельства о регистрации программ для ЭВМ:

1. Раз

2. Два

На основные результаты работы получен патент **RU XXXXXX XX** “Способ построения карты проходимости в окрестности высокоавтоматизированного беспилотного грузового транспортного средства”

Методология и методы исследования.. В диссертации используются методы вероятностной робототехники, методы байесовской оценки, вычислительной оптимизации, а также численный и натурный эксперимент.

Основные положения, выносимые на защиту:

1. Предложенный алгоритм оценки положения объектов в окружающем пространстве ТС по изображению с монокулярной камеры позволяет на 2.7% увеличить коэффициент Жаккара по сравнению с ближайшим аналогом.
2. Предложенный метод построения карты проходимости пространства, позволяющая выполнять неблокирующие операции сдвига, получения и обновления, позволяет в 13.07 раз уменьшить время обновления карты несколькими сенсорами за один цикл получения данных в одномерном случае
3. Предложенный алгоритм проекции визуальных детекций на изображении на плоскость движения ТС на основе комплексирования данных с изображения и облака точек лазерного дальномера позволяет на ?% увеличить коэффициент Жаккара по сравнению с ближайшим аналогом.
4. Интеграция предложенного алгоритма проекции сегментации проезжей части на изображении на плоскость движения ТС на основе комплексирования данных с изображения и лидарного облака в метод построения карты проходимости пространства позволяет уменьшить PathFinding Cost MSE (PFC-MSE) генерируемой карты проходимости на ?%

Достоверность полученных результатов обеспечивается использованием строгого математического аппарата и методов математической статистики. Помимо этого, достоверность подтверждается результатами вычислительных и натурных экспериментов и внедрением разработанных методов. Полученные результаты находятся в соответствии с результатами, полученными другими исследователями.

Апробация работы. Основные результаты работы докладывались на международной конференции 16-th International Conference on Machine Vision(ICMV 2023, Ереван, Армения)

Личный вклад. Все результаты диссертации, вынесенные на защиту, получены автором самостоятельно. Постановка задач и обсуждение результатов проводилось совместно с научным руководителем.

Публикации. Основные результаты по теме диссертации изложены в 0 печатных изданиях, 0 из которых изданы в журналах, рекомендованных ВАК.

Объем и структура работы. Диссертация состоит из введения, 4 глав, заключения и 0 приложений. Полный объём диссертации составляет 45 страниц, включая 15 рисунков и 0 таблиц. Список литературы содержит 0 наименований.

Глава 1. Картографирование окружающего пространства робота на основе комплексирования данных разной природы

1.1 Модели окружающего пространства робота

1.2 Обзор используемых в колёсной робототехнике датчиков и их ограничения

1.3 Классификация методов комплексирования данных

1.4 Требования к алгоритму картографирования окружающего пространства робота

1.4.1 Метрика оценки качества карты проходимости пространства $PFC - MSE$

1.5 Заключение к главе 1

Глава 2. Алгоритмы проецирования визуальных данных на плоскость движения робота

2.1 Алгоритм проекции визуальных детекций на плоскость движения робота на основе модели L-shape

2.2 Исследование точности алгоритма проекции визуальных детекций на основе модели L-shape

2.3 Алгоритм проекции визуальных детекций на плоскость движения робота на основе комплексирования данных с данными лазерных дальномеров

2.4 Исследование точности алгоритма проекции визуальных детекций на основе комплексирования данных с данными лазерных дальномеров

2.5 Алгоритм проекции сегментации проезжей части на изображении на плоскость движения ТС

2.6 Исследование точности алгоритма проекции визуальных детекций на основе комплексирования данных с данными лазерных дальномеров

2.7 Заключение к главе 2

Глава 3. Программный комплекс построения карты проходимости пространства на основе комплексирования разнородных данных

3.1 Назначение и функциональные возможности

3.2 Структура программного комплекса

3.2.1 Библиотека *occipancy_map_fusion*

Компонент *pointcloud_obstacles_mpp*

Компонент *true3d_cam_detection_mpp*

Компонент *delaunay_true3d_segm_mpp*

Компонент *pc_detection_mpp*

3.2.2 Программа *evo_occipancy_map_fusion*

3.3 Заключение к главе **3**

Глава 4. Неблокирующий алгоритм построения карты проходимости пространства на основе комплексирования разнородных данных

4.1 Introduction

Для решения задачи навигации высокоавтоматизированным транспортным средствам (ВАТС) требуется хранить в памяти модель окружающего пространства. Данная модель должна быть достаточно точной и полной для гарантии качества и безопасности осуществляемого движения. В то же время она должна быть вычислительно эффективной как с точки зрения занимаемой памяти, так и времени вычисления на бортовом вычислителе ВАТС. Для достижения требуемой эффективности разрабатываются модели представления окружающего пространства, фокусирующиеся только на малом подмножестве признаков реального мира, необходимых для качественного решения задачи навигации.

Одной из распространённых моделей в робототехнике является карта проходимости пространства – модель, представляющая пространство как двумерную сетку, каждой клетке которой сопоставляется вероятность нахождения препятствия в пространстве, ограниченной этой клеткой. На сегодняшний день предложено множество алгоритмов построения карты проходимости пространства. Их реализация сравнительно проста в случае, если информация об окружающем пространстве поступает от единственного сенсора. Примеры таких систем представлены в статьях [wang2018single] и [luo2019probability]. Однако для большинства практических задач единственного сенсора недостаточно, из-за ограниченного поля зрения, невозможностью полностью компенсировать зашумлённость входных данных, и общими ограничениями, обусловленными принципом его работы. Так, например, использование камер видимого диапазона невозможно в ночное время суток, а качество работы лидаров снижается в условиях сильных осадков.

Чтобы получить более точную модель окружения и повысить безопасность движения ВАТС, на них устанавливаются несколько сенсоров. Примеры систем с несколькими сенсорами описаны в работах [xu2020data], [rosenband2017inside]. Встречаются как системы,

использующие несколько сенсоров одного типа (например, камеры, как в работе [bertozzi1998experience]), так и комбинирующие различные сенсоры, например:

- лидары+радары. Пример такого робота представлен в [gohring2011radar];
- лидары+камеры [tibebu2021lidar]),
- и другие комбинации.

Алгоритмы построения карты проходимости пространства для таких систем в общем случае генерируют карту достаточно полную и точную, но вынуждены решать задачу комплексирования данных для разрешения противоречий показаний с разных сенсоров. В работе [sasiadek2002sensor] представлена классификация методов комплексирования данных по трем группам:

1. Методы, использующие вероятностные модели. К ним относятся Байесовский вывод (Bayesian reasoning), теория Демпстера — Шафера (англ. evidence theory), робастные методы (англ. robust statistics) и т.д.
2. Методы, использующие метод наименьших квадратов (МНК): фильтр Калмана, методы оптимизации (англ. optimal theory), эллипсы неопределённости (англ. uncertainty ellipsoids) и пр.
3. Методы интеллектуального комплексирования: методы нечёткой логики, нейросетевые и генетические алгоритмы.

В настоящее время продолжается активная разработка алгоритмов каждого из трех типов. Примером актуальных исследований в первой группе – работа [duong2022autonomous], в которой используется метод релевантных векторов (метод машинного обучения, использующий Байесовский вывод) для построения непрерывной карты проходимости. Во второй группе можно выделить работу [9274841], в которой положение динамических препятствий на карте обновляется с помощью фильтра Калмана. Стоит отметить, что в данной работе обновление информации о статических препятствиях происходит с помощью Байесовского вывода, поэтому предлагаемый алгоритм является объединением методов из двух представленных групп. Наконец, не менее активно развиваются алгоритмы третьей группы, одним из которых является модель глубокого обучения BEVFusion, представленная в работе [liu2024bevfusionmultitaskmultisensorfusion].

В данной работе мы предлагаем модификацию алгоритма, относящегося к первой группе – картирование на основе Байесовского вывода с использованием обратной модели измерения. Выбор данного подхода обусловлен простотой мо-

дификации базовой реализации, низкой вычислительной сложностью, а также хорошей масштабируемостью при увеличении количества используемых источников входных данных. Первая версия данного алгоритма была представлена в работе Моравица [Moravec_1988]. В рамках этого алгоритма делается предположение, что каждая вероятность занятости клетки в карте проходимости не зависит от вероятностей занятости других клеток карты. Для каждой клетки вычисляется величина так называемого логарифма шансов:

$$l_{t,i} = \log \frac{p(m_i|z_{1:t}, x_{1:t})}{1 - p(m_i|z_{1:t}, x_{1:t})}, \quad (4.1)$$

где $p(m_i|z_t, x_t)$ – вероятность занятости i -й клетки карты, при условии что к моменту времени t робот прошёл траекторию $x_{1:t}$, и его датчики давали показания $z_{1:t}$. Заметим, что здесь используется обратная вероятность: наступления причины (занятость пространства), при условии наступления его следствия (показания сенсора), чем и обусловлено название алгоритма. С получением новых данных значения в тех клетках, о которых появились новые данные (иными словами, которые попали в область видимости датчика), обновляются по следующей формуле:

$$l_{t,i} = l_{t-1,i} + \log \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}, \quad (4.2)$$

где $l_{0,i} = \log \frac{p(m_i)}{1 - p(m_i)}$ логарифм шансов, посчитанный для априорной вероятности занятости клетки. Зачастую, о карте проходимости не делается априорных предположений и $p(m_i)$ устанавливают равным 0.5, что приводит к $l_{0,i} = 0$.

Такой формат обновления удобен, так как позволяет независимо друг от друга посчитать значение $\log \frac{p(m_i|z_t, x_t)}{1 - p(m_i|z_t, x_t)} - l_{0,i}$, на основании полученных данных от каждого сенсора, а затем просто добавить полученное значение к уже имеющейся карте проходимости (при условии атомарности данной операции).

Отметим, что в оригинальной реализации алгоритма предполагается статичность наблюдаемой роботом сцены: клетки, занятые препятствием, не могут быть освобождены позднее без последующих измерений.

кажется это не так, оригинальный алгоритм умеет перетирать место.

Часто реальные условия эксплуатации ВАТС не такие, в них присутствует множество динамических объектов: людей, машин и т.д. Чтобы сделать возможным работу алгоритмов построения карты проходимости пространства для таких сред требуется введение дополнительных техник (см. раздел 4.2).

Как уже было отмечено для построения полной и точной карты проходимости пространства необходимо комплексировать информацию от большого числа сенсоров. Однако, с ростом числа источников данных увеличивается задержка времени обновления карты – интервал между моментом получения данных и их учётом в карте. Определим, чем именно вызвана данная задержка. Алгоритм добавления данных на карту проходимости пространства можно разделить на два шага:

В этом месте очень помогла бы картинка, описывающая последовательность шагов/систему

1. Преобразование входных данных к единому формату представления данных о проходимости пространства. Как правило, это фрагмент карты проходимости, содержащий вероятности занятости клеток, основываясь на последних показаниях данного источника данных.
2. Комплексирование полученных фрагментов с ранее построенной картой проходимости пространства. Для обратной модели измерения этот шаг включает в простом суммировании полученного фрагмента с соответствующей областью карты.

Вычисление первого шага может быть выполнено с помощью асинхронного программирования

что такое параллельный алгоритм? надо переформулировать

, так как для построения фрагмента используются данные только одного источника данных, и синхронизация с другими источниками/подсистемами не требуется. Этот шаг может быть «дорогим» с точки зрения времени выполнения в зависимости от сложности используемого алгоритма, но мы считаем, что уменьшение этого времени трудно реализуемо, так как требует индивидуального анализа и подхода к оптимизации для каждого уникального источника данных. В то же время второй шаг, как правило, является блокирующим: созданный фрагмент добавляется в очередь и включается в карту только после того, как были добавлены все фрагменты, стоящие раньше в очереди. Таким об-

разом происходит задержка, даже в случае добавляемые фрагменты не имеют пересечений и параллельное обновление теоретически возможно.

Тут были комментарии про круговые диаграммы, и оценку задержки от числа сенсоров.

Целью данной работы является предложить асинхронный алгоритм построения карты проходимости пространства, который позволяет снизить итоговое время задержки обновления карты.

Основной вклад данной работы заключается в следующем:

1. Предложены три последовательные модификации базового алгоритма картирования на основе Байесовского вывода с использованием обратной модели измерения для случая одномерного пространства.
2. Проведено сравнение вычислительной сложности базовой реализации и модификаций для всех выполняемых операций над картой проходимости пространства (и их комбинации) при различных размерах картируемой области. Доказано, что применение всех трёх модификаций имеет лучшую вычислительную сложность, чем базовая реализация.

Далее статья организована следующим образом: в разделе 4.2 мы формулируем постановку задачи построения карты проходимости, описываем операции производимые над ней и описываем их базовую реализацию. В следующем разделе 4.3 мы предлагаем модификации базовой реализации, позволяющие уменьшить вычислительную сложность операций, для упрощения восприятия в данном разделе мы упростим задачу и будем рассматривать карту, как одномерное пространство. Далее в разделе 4.4 представлены эксперименты со сравнением вычислительной сложности всех представленных модификаций и исходной реализации для каждой операции в отдельности и их комбинации. В разделе 4.5 мы приводим обобщение последней модификации, показавшей лучшую вычислительную сложность, на случай двумерного пространства. В последнем разделе 4.6 проводится анализ полученных результатов.

4.2 Постановка задачи

Пусть задана фиксированная относительно Земли система координат, расположенная таким образом, что движение робота происходит в плоскости $z = 0$, положение робота в этой системе координат задаётся тремя параметрами: $\mathbf{x} = x, y, \theta$, где x, y координаты фиксированной точки робота (см. рис 4.1),

(1) был робот, а потом стала машина. (2) А точно ли корректно координаты называть параметрами? (3) кажется ссылку на рисунок надо дать где-то здесь сразу

1,3 – fixed. 2 в твоём диссере это были параметры: "В общем случае вектор состояния, которым описывается положение робота определяется шестью параметрами $\mathbf{x} = (x, y, z, \dots$

θ – угол поворота робота от Ox вокруг вертикальной оси. Введём также локальную систему координат робота, начало которой находится в той же фиксированной точке x, y , ось Ox направлена вдоль оси движения робота. Координаты в этой системе координат, мы будем обозначать x_l, y_l . Наконец, пусть у нас есть S источников данных. Каждый источник имеет собственную систему координат, положение которой известно и фиксировано относительно локальной системы координат робота. Источник s независимо от остальных источников генерирует информацию о занятом и/или свободном пространстве в окрестности робота в виде наборов $(P_{s,k}, \mathbf{x}_{s,k}, t_{s,k})$, $s = \overline{1, S}$, $k \in \mathbb{N}$, где $P_{s,k}$ – фрагмент карты проходимости в виде двумерного массива логарифмов шансов занятости пространства с фиксированным разрешением ppm_s (pixel per meter, пикселей на метр), оси массива ориентированы вдоль осей системы координат источника s ; $\mathbf{x}_{s,k}$ – положение начала системы координат сенсора относительно левого верхнего угла массива;

знаечение x я без картинки не понял

Нужно ли перефразировать или обновленного рисунка достаточно?

$t_{s,k}$ – время регистрации сгенерированного фрагмента, k – порядковый индекс сгенерированного набора.

На основе получаемых данных необходимо сформировать локальную карту проходимости пространства в виде набора $(M_i, \mathbf{x}_i, t_i)$; $i \in \mathbb{N}$, где M_i – карта

проходимости пространства в момент времени t_i виде двумерного массива логарифмов шансов занятости пространства с фиксированным разрешением ppm ;
 ppm лучше все-таки везде руссифицировать

Я думал мы хотим это в иностранный журнал подавать? Или мы для кирпича сразу готовимся? Не разорвет ли нас на два фронта воевать?

$\mathbf{x}_i$ — координаты левого верхнего угла карты, заданные таким образом, чтобы начало локальной системы координат в момент времени t_i совпадало с центром карты M_i , оси карты параллельны осям глобальной системы координат. Такая ориентация выбрана, чтобы не вносить погрешность в оценку положения свободного/занятого пространства в результате многократных поворотов на малую величину двумерного массива, которая возникала бы при ориентации карты параллельно осям локальной системы координат.

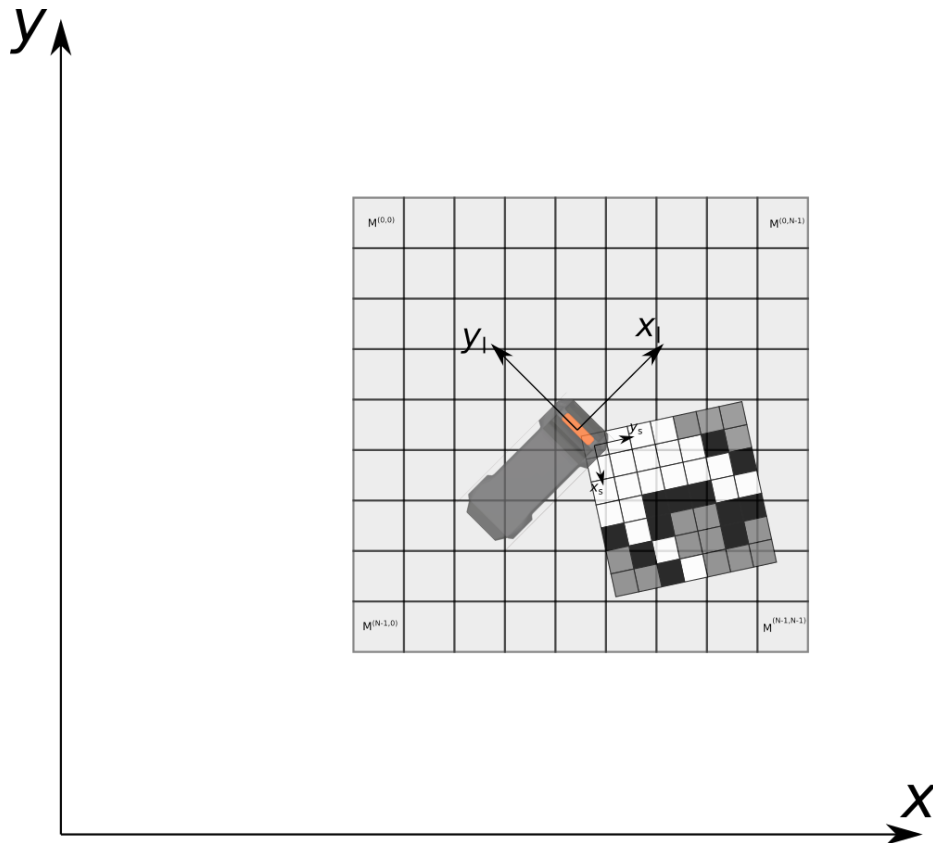


Рисунок 4.1 — Расположение локальной карты проходимости и её фрагмента от одного из источников данных относительно робота и глобальной системы координат

Для построения локальной карты проходимости пространства все генерируемые фрагменты P_s карты проходимости пространства добавляются в очередь в порядке их готовности. Заметим, что время регистрации добавляемых таким образом фрагментов в общем случае не является монотонным (см.

рис 4.2). Далее локальная карта проходимости последовательно обновляется добавлением фрагментов из очереди. Алгоритм одной итерации обновления представлена листинге 1: если $t_{s,k} > t_i$, карта актуализируется к моменту времени фрагмента (строки 3 – 6): вычисляются новые координаты левого верхнего угла карты, время карты заменяется временем регистрации извлечённого фрагмента, содержимое карты сдвигается и «затухает» (механизм и цель «затухания» мы опишем далее в этом разделе). В противном случае значения x_i, t_i остаются неизменными: x_{i-1}, t_{i-1} (строки 8 – 9). Далее извлечённый фрагмент преобразуется: «доворачивается» до ориентации локальной карты проходимости пространства и масштабируется до разрешения локальной карты проходимости, создавая временное представление $P_{aligned}$. Также вычисляется сдвиг левого верхнего угла фрагмента относительно левого верхнего угла локальной карты проходимости пространства $patch_shift$ (строка 12). Если время регистрации фрагмента меньше времени карты, к фрагменту применяется «затухание» (строка 13). Наконец фрагмент добавляется к соответствующей ему области на локальной карте проходимости пространства (строка 14).

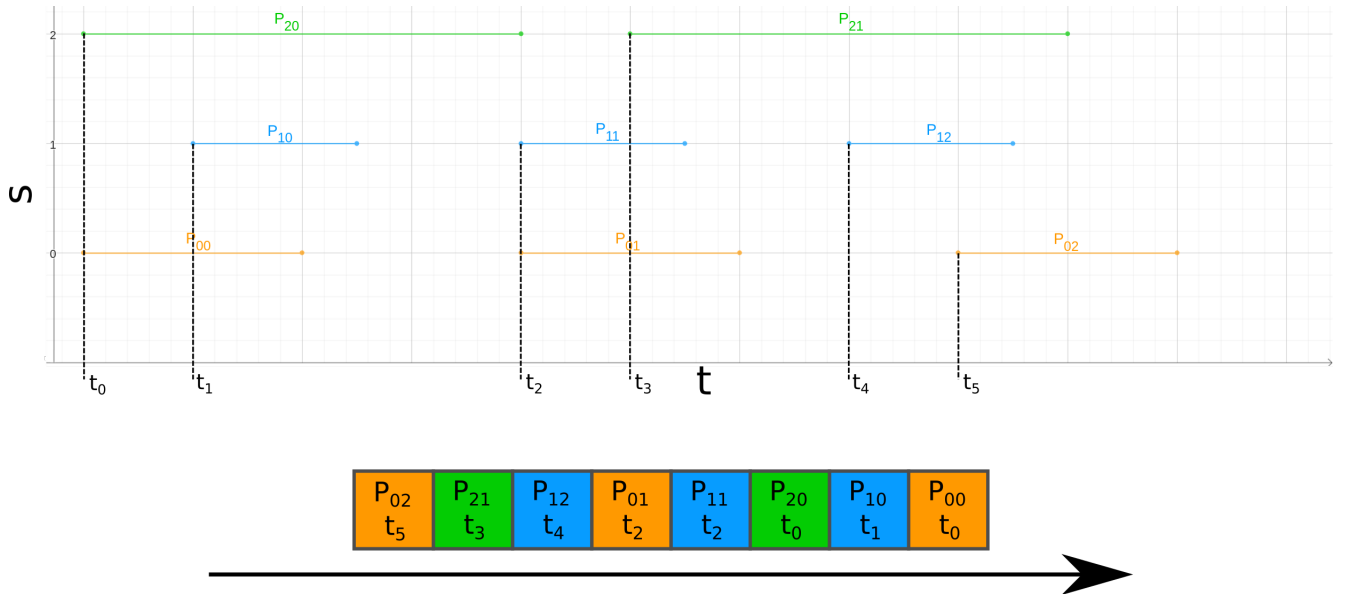


Рисунок 4.2 — Демонстрация формирования очереди фрагментов. Горизонтальные отрезки обозначают время формирования фрагмента каждым источником: начало отрезка соответствует времени регистрации данных $t_{s,k}$, конец отрезка соответствует времени завершения формирования фрагмента и момент добавления его в очередь фрагментов. Внизу иллюстрации продемонстрирован итоговый порядок фрагментов в очереди: фрагменты извлекаются из очереди в порядке справа налево

Algorithm 1 `update_map`

```

1:  $P_{s,k}, x_{s,k}, t_{s,k} \leftarrow \text{pop PatchQueue}$ 
2: if  $t_{s,k} > t_{i-1}$  then
3:    $x_i \leftarrow \text{get\_map\_pose}(t_{s,k})$ 
4:    $t_i \leftarrow t_{s,k}$ 
5:    $M_{actual} \leftarrow \text{shift\_map}(M_{i-1}, x_i - x_{i-1})$ 
6:    $M_{actual} \leftarrow \text{dim}(M_{actual}, t_i - t_{i-1})$ 
7: else
8:    $x_i \leftarrow x_{i-1}$ 
9:    $t_i \leftarrow t_{i-1}$ 
10:   $M_{actual} \leftarrow M_{i-1}$ 
11: end if
12:  $P_{aligned}, \text{patch\_shift} \leftarrow \text{align\_patch}(P_{s,k}, t_{s,k}, t_i, \text{ppm}/\text{ppm}_s)$ 
13:  $P_{aligned} \leftarrow \text{dim}(P_{aligned}, t_i - t_{s,k})$ 
14:  $M_i \leftarrow \text{add\_patch}(M_{actual}, P_{aligned}, \text{patch\_shift})$ 

```

Недостатком обратной модели является предположение о статичности сцены: в случае наличия динамических объектов в сцене, такие объекты оставляют за собой «след», который не удаляется с локальной карты проходимости пространства, если нет источника данных способного отмечать области в которых препятствия отсутствуют. Даже при наличии такого источника, «след» может попасть в его слепую зону и остаться на карте. Для решения данной проблемы, мы вводим механизм «затухания» – показания каждой клетки экспоненциально затухают, возвращаясь в состояние "неизвестно" без повторных обновлений от источников данных (см. листинг 2)

Algorithm 2 $\text{dim}(\widehat{M}_{i-1}, \Delta t)$

```

1:  $a \leftarrow \text{some float-number hyper-param}$ 
2:  $\widehat{M}_i \leftarrow \text{2D array of size: rows}(\widehat{M}_{i-1}) \times \text{cols}(\widehat{M}_{i-1})$ 
3: for  $j = 0$  to  $\text{rows}(M)$  do
4:   for  $k = 0$  to  $\text{cols}(M)$  do
5:      $\widehat{M}_i^{j,k} \leftarrow \exp(-a\Delta t)\widehat{M}_{i-1}^{j,k}$ 
6:   end for
7: end for
8: return  $\widehat{M}_i$ 

```

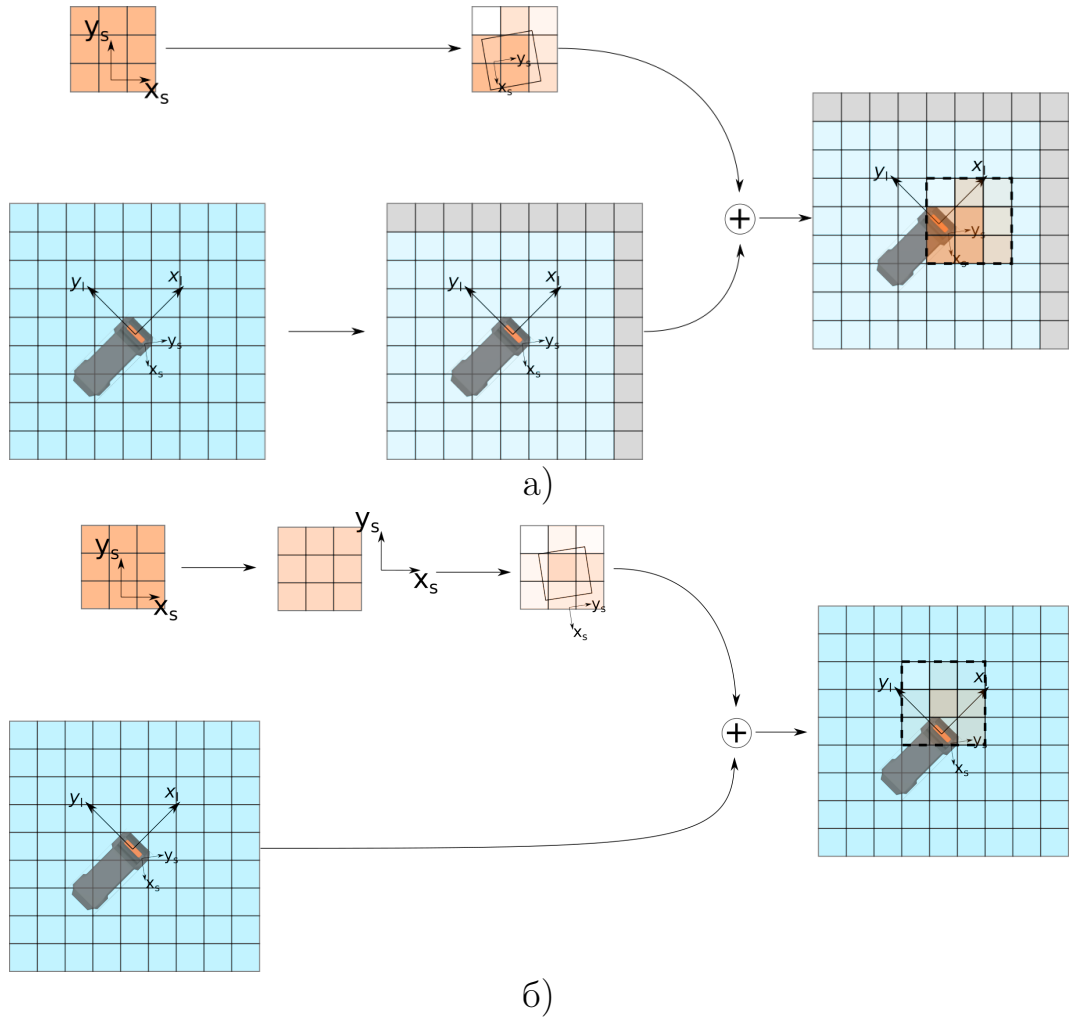


Рисунок 4.3 — Иллюстрация алгоритма обновления локальной карты проходимости пространства фрагментом от источника данных s . 4.3а: время регистрации фрагмента новее последнего времени обновления карты. 4.3б: Сценарий, когда время регистрации фрагмента старше последнего времени обновления карты

Также на шаге добавления фрагмента на карту мы вводим механизм против "перезарядки" клеток карты: если динамическое препятствие долго стояло неподвижно, а потом сдвинулось, то логарифм шансов освободившегося пространства останется отрицательным, и даже с механизмом затухания будут долго оставаться в состоянии «занято». Чтобы этого избежать, мы ограничиваем абсолютную величину максимального значения, которое может принимать клетка:

$$l_t = \max(\min[\text{inv_mes_model}(t), l_{thresh}], -l_{thresh}),$$

где $\text{inv_mes_model}(t)$ – значение логарифма шансов, полученное по формуле 4.2, l_{thresh} – пороговое значение логарифма шансов по абсолютному значению.

4.3 Модификации базовой реализации

В данном разделе мы предлагаем модификации базовой реализации для снижения задержки между появлением информации о проходимости у одного из источников данных и её добавлением на итоговую карту проходимости пространства. Для простоты восприятия, мы упростим задачу и будем считать пространство одномерным: карта в таком случае представляет собой одномерный массив M размера N , а движение возможно только влево или вправо. Также для простоты мы будем считать размер одной клетки равным 1 метру, чтобы опустить операции с учётом разрешения карты ppm . В прошлом разделе мы показали, что карта должна поддерживать операции сдвига и обновления. Помимо этого мы отдельно добавим к описанию реализаций операцию получения карты, поскольку в модификациях данная операция перестаёт быть тривиальным копированием массива и требует отдельного обсуждения.

4.3.1 Base OGM

Для начала опишем исходную реализацию в одномерном случае. Карта представляет собой массив M размера N , текущую координаты центра x_i и время t_i , которому текущая карта соответствует (рис. 4.4).

Сдвиг

При сдвиге в новую точку x_{i+1} , в момент времени t_{i+1} массив со значениями логарифма шанса проходимости сдвигается на $\lfloor x_{i+1} - x_i \rfloor$ клеток. Новые клетки инициализируются 0. Ниже приведён код данной операции. Вычис-

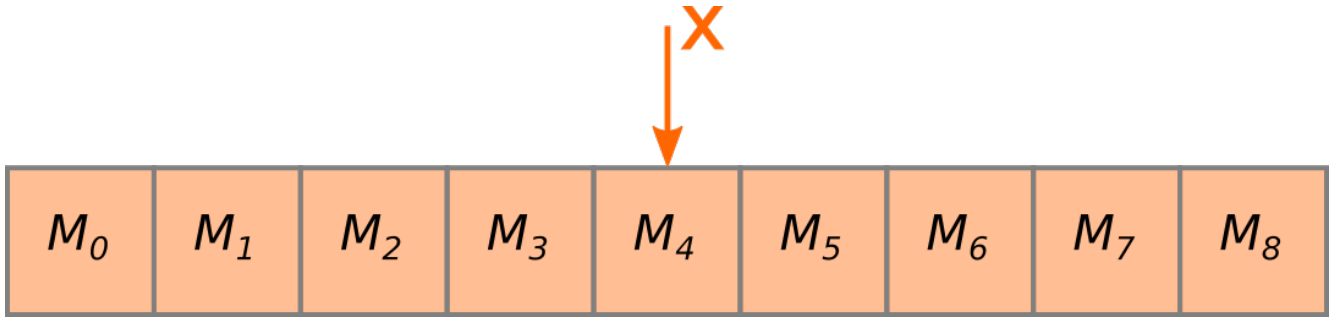


Рисунок 4.4 — Оригинальная реализация локальной карты проходимости пространства

лительная сложность такой операции $\Theta(N)$. Итоговая реализация алгоритма представлена в листинге 3.

Algorithm 3 *shift_map*(M, x_{i+1}, t_{i+1}) (base 1D case)

```

1:  $\Delta x \leftarrow \text{round}(x_{i+1} - x_i)$ 
2:  $j \leftarrow 0$ 
3: for  $j = 0$  to  $N$  do
4:   if  $j + \Delta x \in [0; N)$  then
5:      $M[j] \leftarrow M[j + \Delta x]$ 
6:   else
7:      $M[j] \leftarrow 0$ 
8:   end if
9: end for
10: return  $M$ 

```

Обновление

Процесс обновления в одномерном случае во многом повторяет алгоритм, представленный в 4.2. Отметим только, что в данном случае «доворачивание» фрагмента не требуется, и необходимо только учесть сдвиг карты за промежуток времени между моментом регистрации фрагмента и итоговым временем регистрации карты после обновления. Дополнительно введём обозначение размера фрагмента P : N_P . Итоговая реализация алгоритма представлена в листинге 4.

Algorithm 4 *update_map*($P_s, t_{s,k}, x_{s,k}$) (base 1D case)

```

1:  $x_{t_{s,k}} \leftarrow$  local occupancy map pose at  $t_{s,k}$ 
2: if  $t_{s,k} > t_i$  then
3:    $M \leftarrow \text{shift\_map}(M, x_{t_{s,k}}, t_{s,k})$ 
4:    $M \leftarrow \text{dim}(M, t_{s,k} - t_i)$ 
5: else
6:    $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ 
7:    $P_s \leftarrow \text{dim}(P_s, t_i - t_{s,k})$ 
8: end if
9:  $\Delta x_s \leftarrow \text{round}(x_{s,k} - x_i)$ 
10: for  $j \leftarrow \max(0, \Delta x_s)$  to  $\min(N_P; N + \Delta x_s)$  do
11:    $M[j - \Delta x_s] \leftarrow M[j - \Delta x_s] + P_s[j]$ 
12: end for

```

Первая часть алгоритма (до цикла добавления фрагмента на карту) имеет вычислительную сложность $\Theta(1)$, если $t_{patch} \leq t_M$, и, как показано в предыдущем пункте, $\Theta(N)$ (операция затухания карты имеет ту же вычислительную сложность) в противном случае. Если патч целиком помещается во второй части алгоритма будет выполнено $\Theta(N_P)$ операций. Если же фрагмент больше карты и полностью её покрывает будет выполнено $\Theta(N)$ операций. В то же время вторая часть алгоритма может не выполняться вовсе в случае если фрагмент не пересекается с картой. Таким образом вычислительная сложность второй части алгоритма: $O(\min(N_P, N))$, а суммарная вычислительная сложность всего алгоритма: $O(N + \min(N_P, N))$.

Получение карты

Для получения карты необходимо просто вернуть копию текущего массива. Сложность операции: $\Theta(N)$. Копия требуется для того, чтобы не блокировать последующие операции обновления и сдвига. В силу тривиальности алгоритма мы не приводим здесь его реализацию.

4.3.2 OGM with prefetched areas

Первая модификация локальной карты проходимости пространства направлена на снижение стоимости сдвига. Для этого массив для хранения карты увеличивается до размера ($\hat{N} > N$), чтобы описывать область больше, чем отображаемая карта.

В такой реализации к уже упомянутому массиву и координате x добавляется координата, соответствующая некоторой фиксированной точке "полного" массива $\hat{x}$. Возьмём в качестве такой точки центр массива (Рис. 4.5).

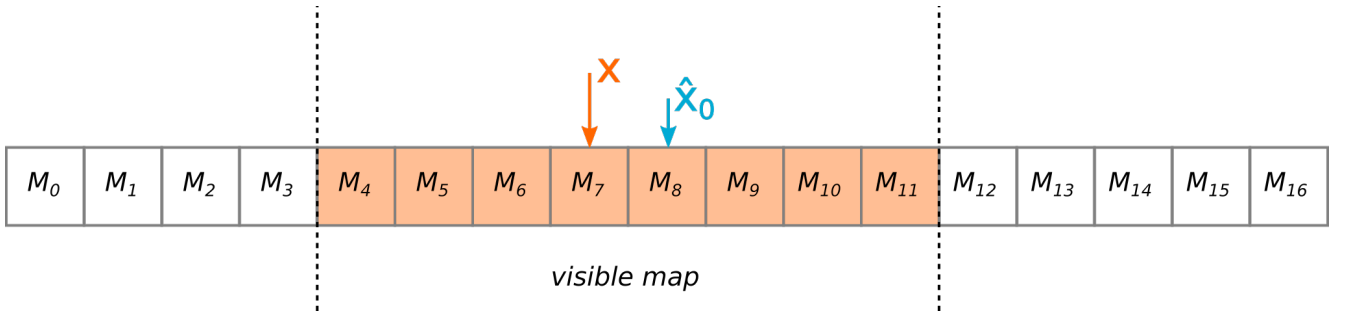


Рисунок 4.5 — Карта с подготовленными окрестностями

Сдвиг

После описанной модификации для сдвига, в общем случае будет достаточно только обновить значение x , соответственно эта операция требует константное время для исполнения $\Theta(1)$. Если же при сдвиге "отображаемая" часть карты выходит за границы полного массива, его содержимое сдвигается так, чтобы новый $\hat{x}$ совпал с актуальным x . В этом случае сложность, как и в оригинальной реализации, $\Theta(N)$. Как итог, при правильно выбранном размере массива, амортизационная сложность операции: $\Theta(1)$.

Algorithm 5 *shift_map*(M, x_{i+1}, t_{i+1}) (prefetched)

```

1: if  $(\hat{x} - x_{i+1}) \in [N/2; \hat{N} - N/2)$  then
2:   return M
3: end if
4: ...
5:           ▷ Previous map_shift implementation but for whole prefetched map
6: ...

```

Обновление

Операция обновления карты в точности повторяет обновление из оригинальной реализации. Сложность алгоритма: $O(\min(N_P, N))$, если не происходит затухания карты и $O(\hat{N} + \min(N_P, N))$ в противном случае (мы должны учитывать клетки за границами видимой карты, т.к. они могли быть ранее обновлены и вернуться в область видимости в будущем).

Получение карты

Операция получения карты заключается в копировании подмассива, соответствующего видимой части карты. Как и прежде, сложность операции – $O(N)$. Все операции по-прежнему остаются блокирующими.

4.3.3 OGM with lockfree cell value update

Блокировки обновления карты становятся узким местом алгоритма при увеличении числа источников данных, используемых для заполнения карты. Каждый источник вынужден ждать своей очереди для доступа к карте, даже если добавляемые фрагменты не пересекаются между собой. Чтобы снизить задержку в таком случае, мы предлагаем вторую модификацию локальной карты проходимости пространства, изменяющую структуру самой клетки карты.

Ранее клетка содержала одно число с плавающей точкой. Теперь это будет атомарная пара чисел: значение логарифма шансов занятости клетки и время, которому оно соответствует (таким образом, мы более не требуем, чтобы все клетки массива соответствовали одному моменту времени). Заметим, что при разработке атомарной структуры данных (т.е. структуры данных, не использующей примитивы блокировки: мьютексы, семафоры, сигналы и т.п. при чтении/записи) важно учитывать её размер и функциональные возможности аппаратного обеспечения. В нашем случае структура состоит из двух 32-битных полей и имеет итоговый размер – 64 бита. Большинство современных архитектур имеют инструкции, обеспечивающие атомарные операции с такими структурами данных.

Сдвиг

Операция сдвига не претерпела значительных изменений в сравнении с предыдущей реализацией, с тем уточнением что при копировании клеток во время «полноценного» сдвига копируется не только значение логарифма шансов, но и время, а также в данной реализации необходимо инициализации времени клетки (мы используем значение 0, т.к. оно не влияет на последующие операции обновления клетки). Сложность операции остаётся амортизированной $\Theta(1)$ и в худшем случае имеет сложность $\Theta(\hat{N})$. Заметим однако, что реальная константа сложности увеличится из-за использования атомарных структур, копирование и инициализации, которых значительно продолжительнее своих неатомарных аналогов.

Обновление

При обновлении карты многие шаги из предыдущего раздела повторяются. Сперва мы определяем необходим ли сдвиг карты. Если нет, обновление может быть выполнено одновременно с другими такими же операциями обнов-

ления, не требующими сдвига. В противном случае операция ждёт возможности взять уникальные права на модификацию карты и произвести требуемый сдвиг.

Далее мы корректируем координаты, куда требуется добавить фрагмент карты проходимости пространства. Для каждой клетки мы атомарно считываем её значения, аналогично предыдущим версиям алгоритма пересчитываем новое значение клетки с учётом «затухания» и пытаемся атомарно записать в клетку, если её значение не изменилось за то время, что мы вычисляли новое значение клетки (сравнение с обменом, Compare and Swap, CAS). Если клетка поменяла своё значение, мы пересчитываем новое значение клетки с учётом обновившегося состояния обновляемой клетки до тех пор, пока не произведём успешную запись. Листинг описанного алгоритма приведён в листинге алг. 6

Теоретически данная операция может выполняться неограниченно долго, если на между попытками сравнения с обменом происходит обновление клетки в другой операции обновления. На практике незавершаемость операции практически невозможна ввиду в первую очередь ограниченного числа потоков, работающих одновременно (в нашем случае их 12, на современных вычислителях используемых в робототехнике их число варьируется от 8 до 24). Если пренебречь единичными пересчётами обновляемых значений, сложность такой операции становится $O(\min(N_P, N))$ ($O(\hat{N} + \min(N_P, N))$), если во время выполнения операции произошёл «полноценный» сдвиг карты). Однако константа в этом случае будет заметно больше, чем в предыдущих вариантах, из-за использования а) атомарных операций, которые в несколько раз вычислительно дороже, неатомарных аналогов и б) `shared_mutex`, который имеет значительно большее время блокировки в сравнении с обычным `mutex`-ом. Поэтому эта реализация становится выгоднее предыдущих вариантов при условии достаточного количества источников производящих одновременное обновление карты.

Получение карты

При получении карты, копируются все клетки видимой части карты, чтобы избежать состояния гонки с другими операциями. В то же время находится самое позднее значение времени в клетках t_{newest} . После чего создаётся карта «старого» формата – массив чисел с плавающей точкой, куда записываются

Algorithm 6 *update_map*($P_s, t_{s,k}, x_{s,k}$) (partially lockfree)

```

1:                                     ▷ Enter section with allowed simultaneous update
2: shared_lock(map_access_mutex)
3:  $x_{t_{s,k}} \leftarrow$  local occupancy map pose at  $t_{s,k}$ 
4: if (Shift is required) then                                     ▷ Critical section start
5:   unique_lock(map_access_mutex)
6:   if Shift is still required and was not performed by other update then
7:      $M \leftarrow \text{shift\_map}(M, x_{t_{s,k}}, t_{s,k})$ 
8:   else
9:      $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ 
10:  end if
11: else                                                             ▷ Critical section end
12:    $x_{s,k} \leftarrow x_{s,k} - (x_{t_i} - x_{t_{s,k}})$ 
13: end if
14:  $\Delta x_s \leftarrow \text{round}(x_{s,k} - x_i)$ 
15: for  $j \leftarrow \max(0, \Delta x_s)$  to  $\min(N_P; N + \Delta x_s)$  do
16:    $t_{prev} \leftarrow 0$ 
17:    $val_{prev} \leftarrow 0$ 
18:    $t_{new} \leftarrow t_{s,k}$ 
19:    $val_{new} \leftarrow P_s[j]$ 
20:
21:   while CAS( $M[j - \Delta x_s], \{val_{prev}, t_{prev}\}, \{val_{new}, t_{new}\}$ ) failed do           ▷
   Compare and swap values until success.  $\{val_{prev}, t_{prev}\}$  are actualized in case
   CAS fails
22:     if  $t_{prev} < t_{s,k}$  then
23:        $t_{new} \leftarrow t_{s,k}$ 
24:        $val_{new} \leftarrow val_{prev} \exp(-a(t_{s,k} - t_{prev})) + P_s[j]$ 
25:     else
26:        $t_{new} \leftarrow t_{prev}$ 
27:        $val_{new} \leftarrow val_{prev} + P_s[j] \exp(-a(t_{prev} - t_{s,k}))$ 
28:     end if
29:   end while
30: end for

```

значения скопированных клеток, приведённых к моменту времени t_{newest} . Сложность такой операции $O(N)$, однако как и в случае с операцией обновления, константа, скрываемая за O , превышает старое значение. Заметим, что эта операция может выполняться одновременно с операциями обновления локальной карты проходимости пространства, если допускается частично обновлённое состояние карты. Рассмотрим, например, такой сценарий

1. Операция получения карты считала значение $M[0]$
2. Источник данных обновил значения клеток $M[0]$, $M[1]$
3. Операция получения карты считала значение $M[1]$

В этом случае только в $M[1]$ будут учтены показания источника данных. Естественнo в последующих вызовах эти показания будут учтены в обеих клетках. Если такая несогласованность данных считается недопустимой, операция получения карты должна быть блокирующей аналогично сдвигу.

4.3.4 Fully lockfree OGM

В рамках последней модификации мы избавляемся от последнего ограничения карты и делаем операцию сдвига так же не блокирующей. Для этого мы разделим карту на два равных примыкающих друг к другу фрагмента, добавим к каждому координату, описывающую его положение (как и ранее это будет центр фрагмента) и будем хранить их в умных, разделяемых указателях (рис. 4.6) pM_1, pM_2 . Для нашей реализации важно, чтобы аппаратное обеспечение позволяло работать с такими указателями атомарно. Большинство современных архитектур поддерживают атомарные операции для стандартной C++ реализации `std::shared_ptr`. Примем размер одного фрагмента равным $\hat{N}$.

Сдвиг

Прежде чем приступить к описанию операции важно сделать следующее замечание: предлагаемая реализация сдвига является неблокирующей относи-

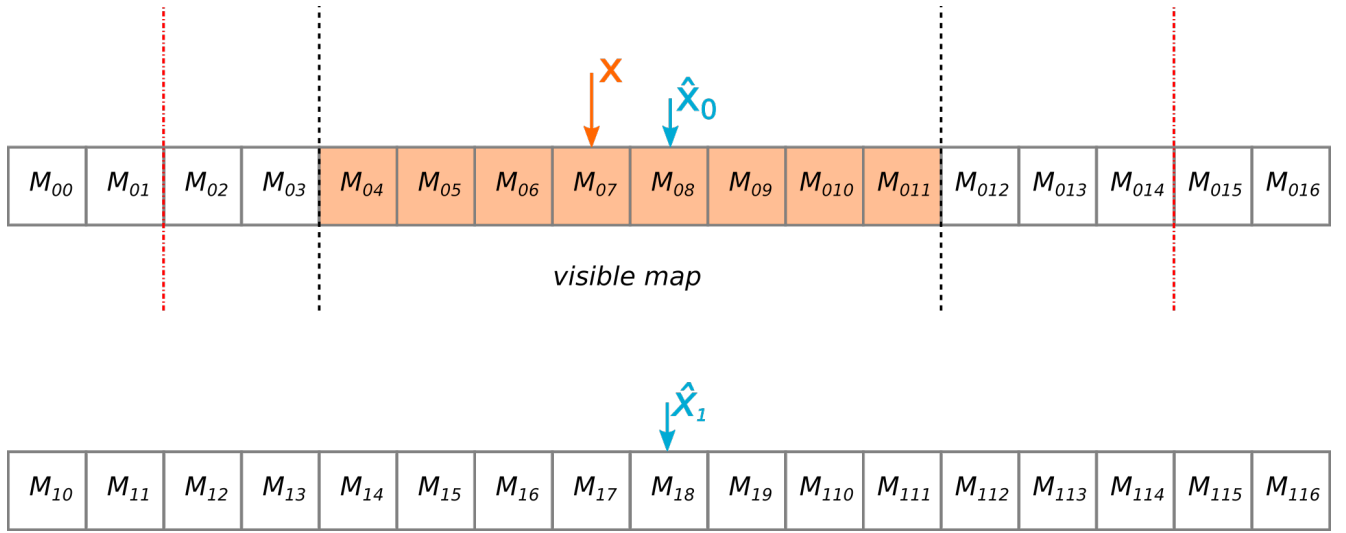


Рисунок 4.6 — Карта с неблокирующей операцией сдвига

тельно обновления и/или получения карты, но в то же время, эта версия не может выполняться одновременно с другими операциями сдвига.

При сдвиге мы определяем положение отображаемой карты после сдвига. Если отображаемая часть карты присутствует целиком на одном фрагменте, и в результате сдвига отдаляется от его центра более заданного порога Δ_{lim} (см. рис. 4.6) и при этом так же отдаляется и от второго фрагмента (проще говоря, если дальнейшее движение карты в том же направлении приведет к выходу за границы всех фрагментов), то тот фрагмент, на котором сейчас не находится карты заменяется на новый: все значения клеток (как и ранее это атомарные пары значения логарифма шансов и времени) инициализируются $\{0,0\}$, а его координата меняется таким образом, чтобы при дальнейшем движении в том же направлении отображаемая часть карты «перешла» на этот фрагмент. В противном случае, как и ранее мы просто обновляем положение отображаемой части карты (см. листинг алг. 7). В данном алгоритме важно правильно подобрать Δ_{lim} таким образом, чтобы отображаемая часть карты не могла выйти за границы фрагмента, когда производится замена второго фрагмента (в противном случае это может повлиять на операцию получения карты, выполняемой одновременно со сдвигом).

Вычислительная сложность, как и ранее, $\Theta(1)$, в общем случае, когда обновляется только координата отображаемой части локальной карты проходимости пространства. $\Theta(\hat{N})$, если происходит замена второго фрагмента.

Algorithm 7 *shift_map*(M, x_{i+1}, t_{i+1}) (fully lockfree)

```

1: if Current map center is between  $M_1$  and  $M_2$  centers then return  $M$ 
2: end if
3:  $M_c \leftarrow$  fragment with current map
4:  $\hat{x} \leftarrow M_c$  center
5: if  $|\hat{x} - x_{i+1}| > \Delta_{lim}$  then
6:    $M_{new} \leftarrow$  new fragment of size  $\hat{N}$ 
7:   for  $i \leftarrow 0$  to  $\hat{N}$  do
8:      $M_{new}[i] \leftarrow \{0, 0\}$ 
9:   end for
10:  Swap pointer to  $M_{new}$  with second fragment (not  $M_c$ )
11: end if return  $M$ 

```

Обновление карты

Обновление происходит аналогично предыдущим реализациям за исключением следующих моментов:

1. В самом начале операции происходит копирование указателей на фрагменты M_1, M_2 , чтобы гарантировать их сохранение во время выполнения операции (на случай если одновременно с этой операцией выполняется сдвиг, заменяющий один из фрагментов).
2. Т.к. операция сдвиг не может выполняться с другими операциями сдвига, в обновлении больше эта операция более не происходит. Вместо этого она либо выполняется в отдельном потоке с определённой частотой (гарантирующей, что отображаемая часть карты всегда содержится на одном или обоих фрагментах карты), либо например происходит в операции получения локальной карты проходимости пространства (если гарантируется, что эта операция не выполняется одновременно с такой же)
3. Операция применяется последовательно к обоим фрагментам карты

Таким образом вычислительная сложность остаётся $O(\min(N_P, N))$. Реальная константа будет незначительно больше, чем в предыдущей реализации из-за атомарного копирования указателей и применения на двух фрагментах

(заметим что последнее может быть выполнено одновременно, но асимптотику операции в общем случае это не изменит).

Получение карты

Аналогично обновлению, операция получения почти не отличается от предыдущей реализации, за тем исключением, что первым действием операция копирует указатели, чтобы гарантировать сохранность данных во время работы. Вычислительная сложность, как и раньше $O(N)$.

4.4 Сравнение задержки обновления локальной карты проходимости пространства

Чтобы сравнить представленные модификации между собой мы подготовили их реализации на C++ и синтетические тесты для измерения времени их работы. В рамках этих тестов симулируются следующие сценарии:

- Сдвиг карты на 20 клеток.
- Добавление фрагмента на карту в одном потоке. Размер фрагмента фиксирован и равен 100. Положение фрагмента меняется и берётся циклично из списка: $-100, -50, -25, -1, 0, 1, 5, 10, 25, 50, 100$. Таким образом моделируются сценарии в которых только часть фрагмента попадает на локальную карту проходимости пространства.
- Добавление фрагмента на карту в нескольких потоках. Тот же сценарий, что и в предыдущем пункте, но запускается в нескольких потоках. Протестировано варианты с разным числом потоков: 1, 5, 9, 13.
- Получение локальной карты проходимости пространства.
- Стандартный цикл "публикации" локальной карты проходимости пространства: В 8 потоках производится сдвиг карты на 10 клеток и добавления патча размера 100, после завершения работы всех потоков, вызывается операция получения карты, имитирующая передачу карты потребителям.

Все эксперименты повторялись для "видимых областей" разного размера N : 10, 16, 32, 64, 128, 256, 516, 1024, 2048, 4000. Для модификаций, мы брали значение $\hat{N} = 2N$ (в случае с последней модификации с полностью неблокирующими операциями полный размер карты с учётом неотображаемых областей составил $4N$)

4.4.1 Сдвиг карты

Результаты замеров представлены на рис. 4.7. Можно видеть линейный рост исходной реализации, как и было показано при обсуждении реализации. Остальные реализации имеют константное (в среднем), время выполнения, поскольку в общем случае требуют только обновления координаты x . Тем не менее константы отличаются между собой. Так самое быстрое время выполнения у полностью неблокирующей реализации, в которой не используются мьютексы для создания критических областей. Далее идёт версия карты с запасом

жаргонизм какой-то. Стоит ли мне указывать модификации по порядковым номерам? Или есть способ аккуратнее назвать её?

, которая использует стандартный `std::mutex`. Наконец, худшее время выполнения среди модификаций имеет версия с неблокирующим обновлением, поскольку здесь используется блокировка `std::shared_mutex`. Отметим также что на малых размерах наивная (оригинальная) реализация показывает себя эффективнее модификаций.

4.4.2 Добавление фрагмента в одном потоке

Результаты замеров представлены на рис. 4.8. Как и в прошлом эксперименте оригинальная реализация имеет линейную сложность. Такую же сложность имеет версия карты с запасом. Такое поведение объясняется необходимостью приводить все клетки массива к одному моменту времени. Этим же объясняется большее время, которое выполняется сдвиг карты с запасом в сравнении с оригинальной версией: ей требуется обновить больше число клеток

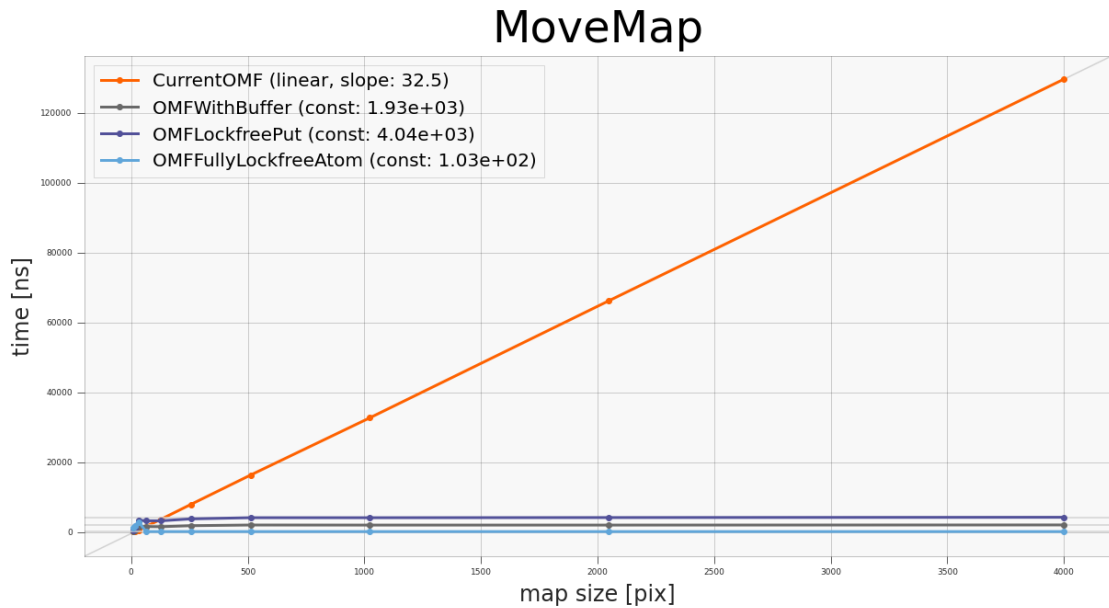


Рисунок 4.7 — График зависимости времени сдвига карты от размера карты.

(вдвое больше, если быть точным, что хорошо отражают вычисленные экспериментально коэффициенты наклона). В то же время оставшиеся модификации с увеличением карты выходят на константу, т.к. не обновляют всю карту целиком, а только те клетки, что пересекаются с фрагментом. При этом у полностью неблокирующей реализации производительность ниже, чем у версии с неблокирующим обновлением. Вероятно это объясняется необходимостью атомарно копировать указатели на фрагменты карты, обращаться к элементам через указатель и повторению операции на двух массивах (последние два фактора снижают локальность данных и приводит к большему количеству промахов кэша).

4.4.3 Добавление фрагмента в нескольких потоках

Результаты замеров представлены на рис. 4.9 4.9 4.10 4.12. Здесь применимы рассуждения из предыдущего пункта. Можно отметить, что с ростом числа потоков уменьшается размер карты, при котором неблокирующие модификации становятся вычислительно эффективнее оригинальной реализации.

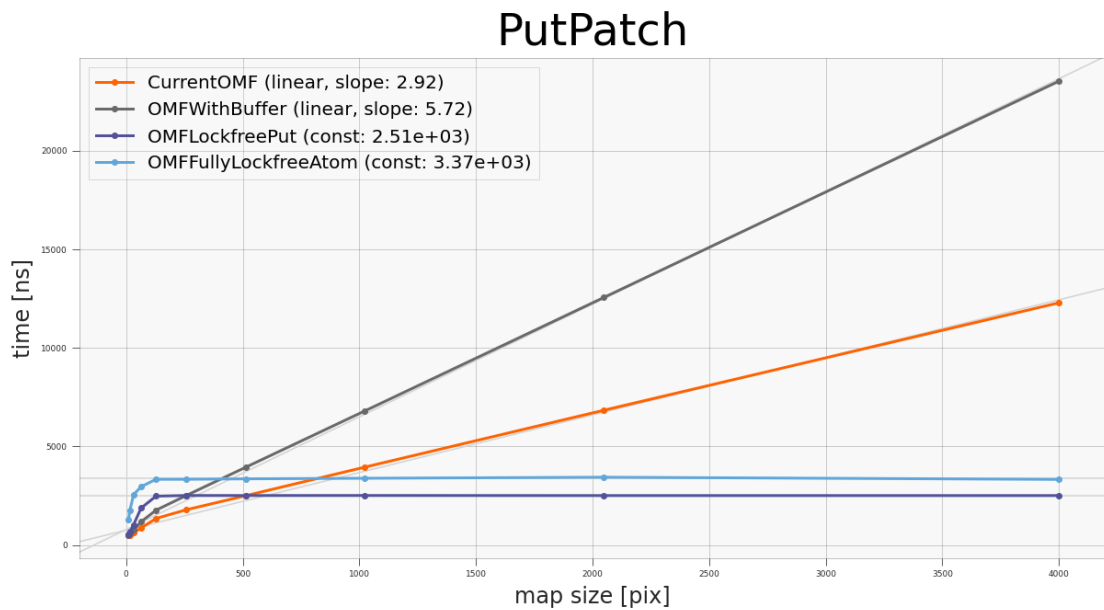


Рисунок 4.8 — График зависимости времени добавления фрагмента на карту от размера карты.

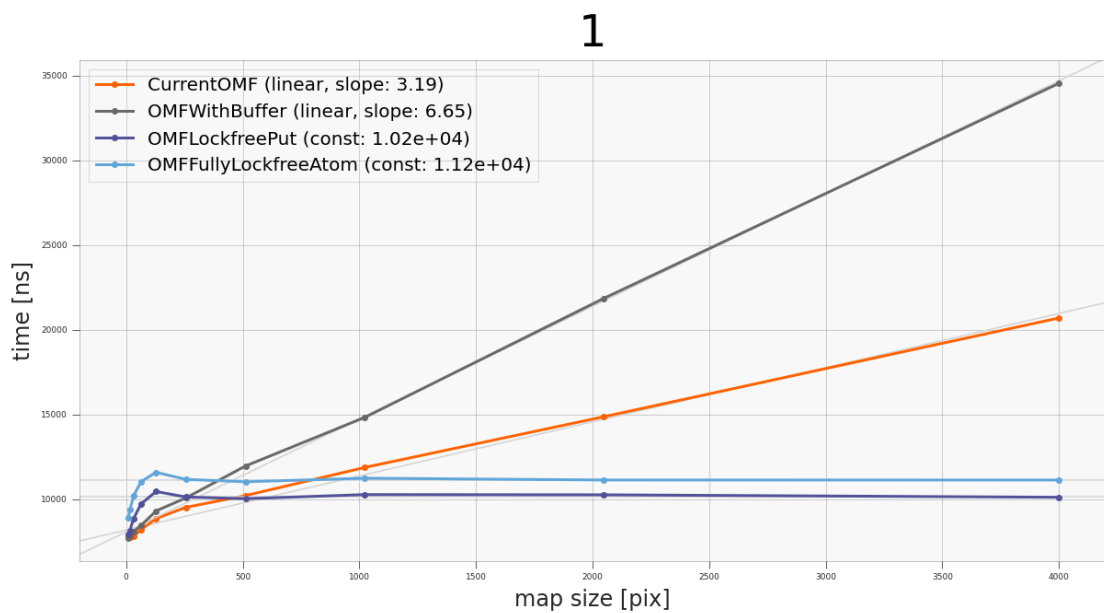


Рисунок 4.9 — График зависимости времени добавления фрагмента на карту в 1 потоке от размера карты.

4.4.4 Получение карты

Результаты замеров представлены на рис. 4.13. Здесь результаты заметно отличаются от предыдущих экспериментов. Время получения оригинальной

5

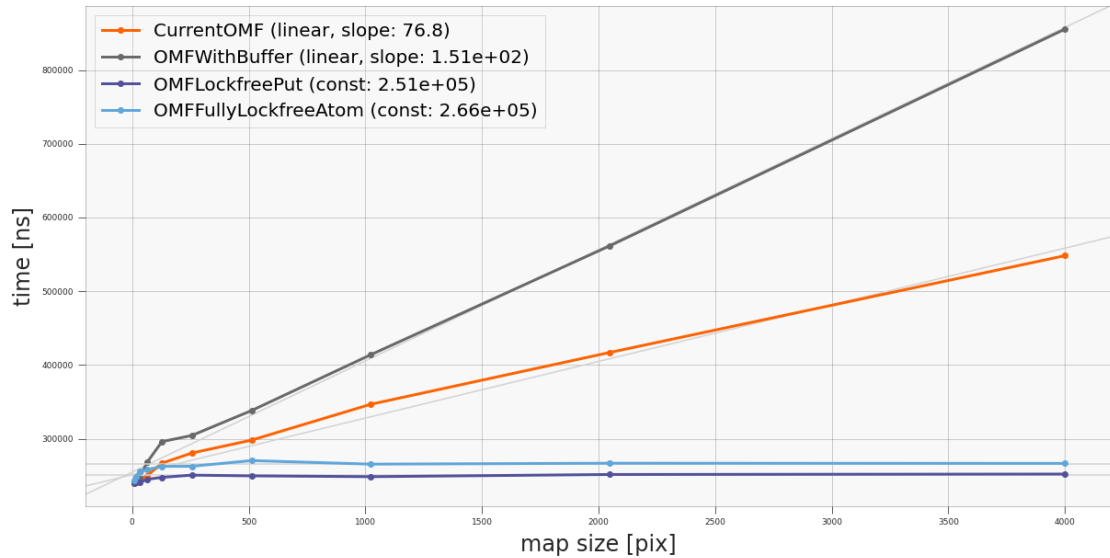


Рисунок 4.10 — График зависимости времени добавления патча на карту в 5 потоках от размера карты.

9

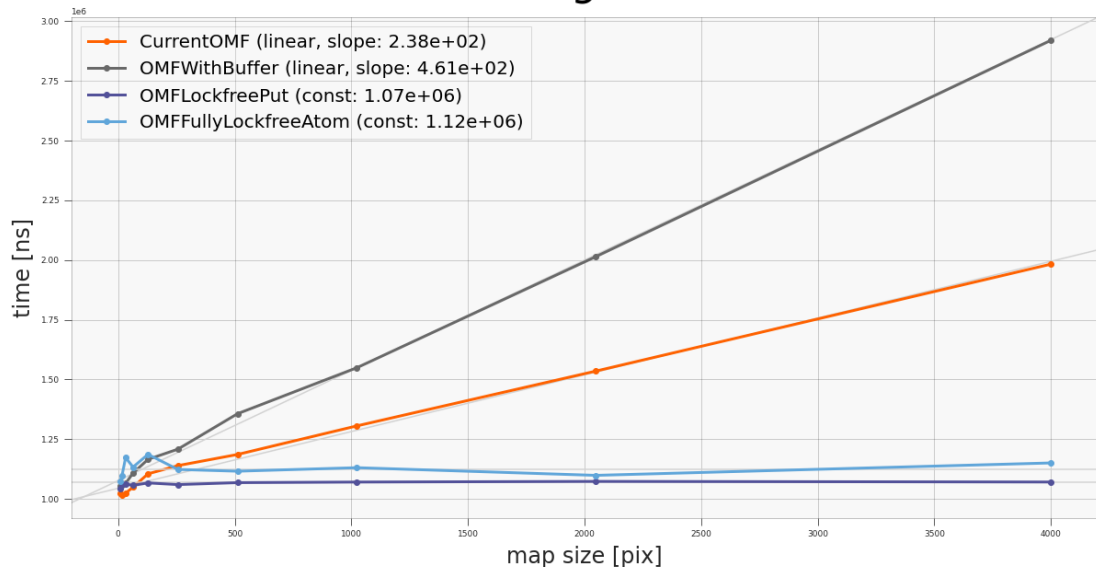


Рисунок 4.11 — График зависимости времени добавления патча на карту в 9 потоках от размера карты.

реализации и карты с запасом с увеличением карты возрастает настолько медленно, что кажется константным в сравнении с неблокирующими реализациями (тем не менее оно линейно)

нужен отдельный график, чтобы это показать?

13

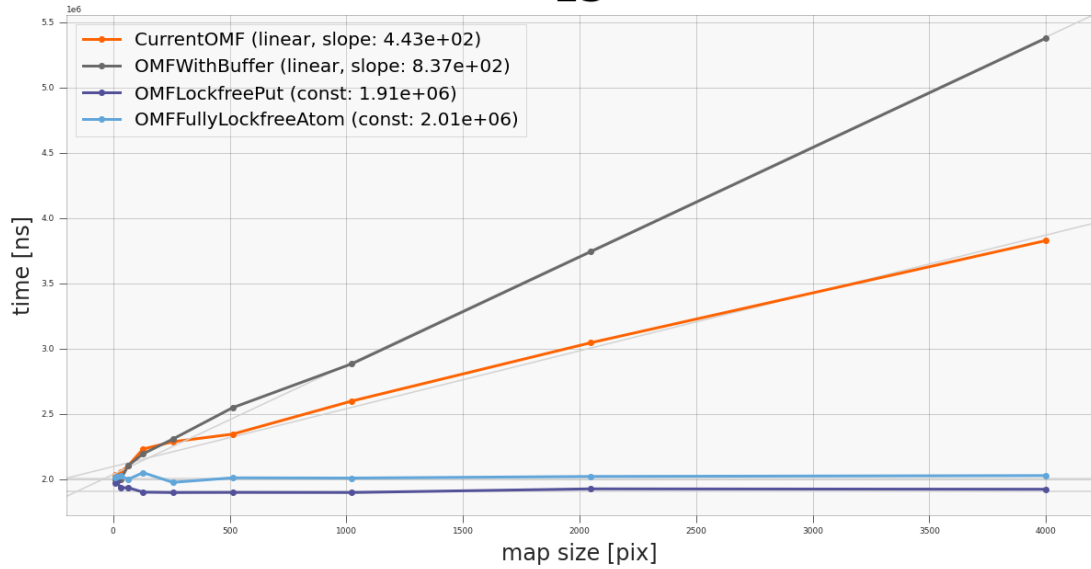


Рисунок 4.12 — График зависимости времени добавления патча на карту в 13 потоках от размера карты.

. В последних же за счёт необходимости атомарно считывать значения клеток, а также приводить их к общему моменту времени, наблюдается линейный рост.

GetMap

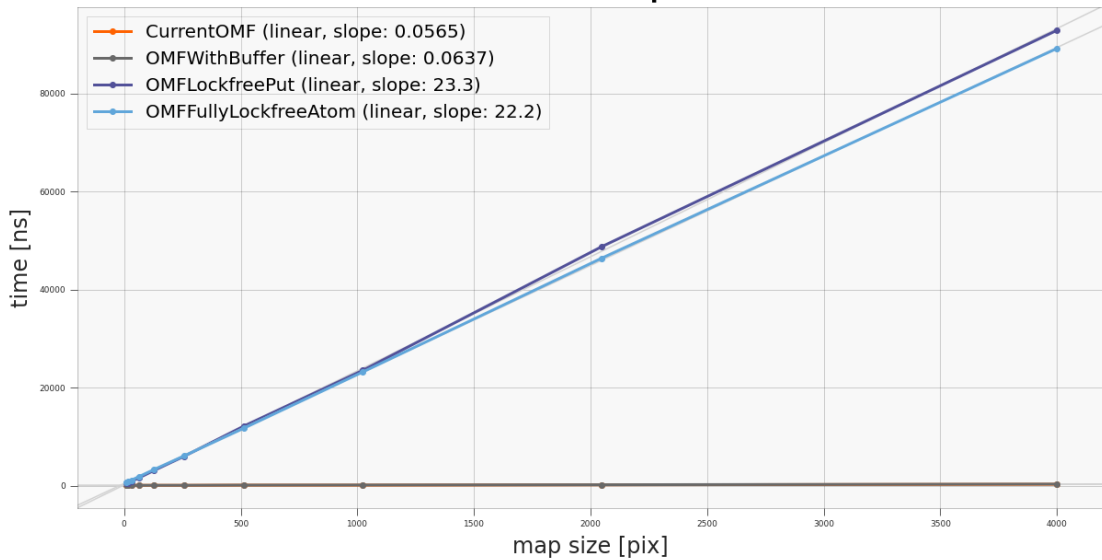


Рисунок 4.13 — График зависимости времени получения карты от её размера. График BaseOGM практически полностью совпадает с OMFWithBuffer и на таком масштабе они не отличимы.

4.4.5 Стандартный цикл "публикации" карты

Наиболее показательная для нас характеристика, показывающая какая реализация покажет себя лучше всего в реальном сценарии использования. Результаты приведены на рис. 4.14. Можно видеть что все представленные реализации имеют линейную вычислительную сложность относительно размера карты. Все представленные модификации превосходят по эффективности оригинальную реализацию, при чём каждая модификация показывает себя лучше предыдущей (по крайней мере по мере увеличения размера карты). Лучшую производительность показала полностью неблокирующая реализация: её экспериментально установленная константа линейной сложности в 13.89 меньше исходной реализации. Далее идёт модификация с неблокирующим обновлением содержимого карты (в 11.52 быстрее оригинальной исходной реализации). Наконец модификация карты с запасом, при всей своей тривиальности показала увеличение производительности в 6.45 раз.

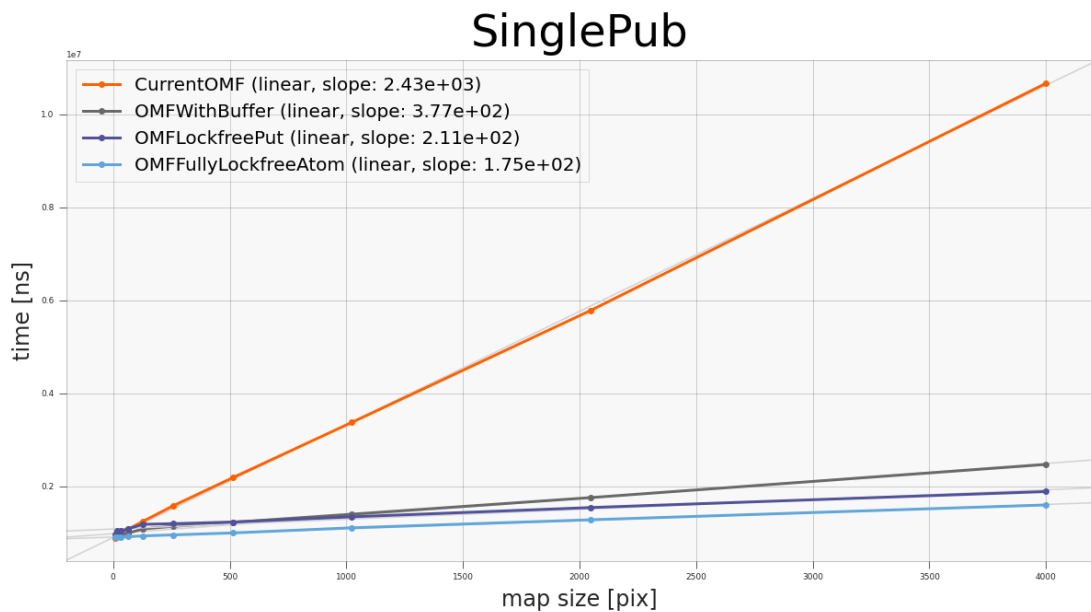


Рисунок 4.14 — График зависимости времени одной итерации "публикации" карты от её размера.

4.5 Реализация для 2D случая

Показав работоспособность неблокирующей модификации локальной карты проходимости пространства для одномерного случая, представим её расширение для двумерного случая. Как и ранее, мы разбиваем полную карту окружения на несколько, теперь уже 2D, массивов, состоящих из уже описанных ранее атомарных пар логарифма шансов и времени. Для движения в плоскости потребуется 9 массивов (рис. 4.15). Если видимая область карты пересекает границы отмеченные штрих-пунктирной линией создаются новые массивы и заменяют часть предыдущих таким образом, чтобы текущий массив, в котором находится видимая область карты, стал центральным. Важно отметить, что отступ этих линий от краёв массива должен быть подобран так, чтобы за одну итерацию сдвига, видимая область не могла преодолеть расстояние между линией и границей массива. В противном случае если параллельно будет происходить получение карты, то массив, в который "переходит" видимая область может быть ещё не загружена, что приведёт к ошибке выполнения.

Легко видеть, что такая реализация значительно увеличивает размер памяти, требуемой для хранения полученной структуры данных. Кроме того, как и в одномерном случае, операции получения и обновления карты должны применяться ко всем 9 массивам. Всё это существенно увеличивает вычислительную сложность. Требуется проведение дополнительных экспериментов для определения условий (средняя скорость движения ВАТС/средний размер добавляемых фрагментов/количество источников данных и частота их работы/масштаб локальной карты проходимости пространства), при которых такая реализация будет вычислительно эффективнее оригинальной версии.

очень надо результаты замеров собрать в одну таблицу и вставить в приложение. Особенно это важно для самого диссера

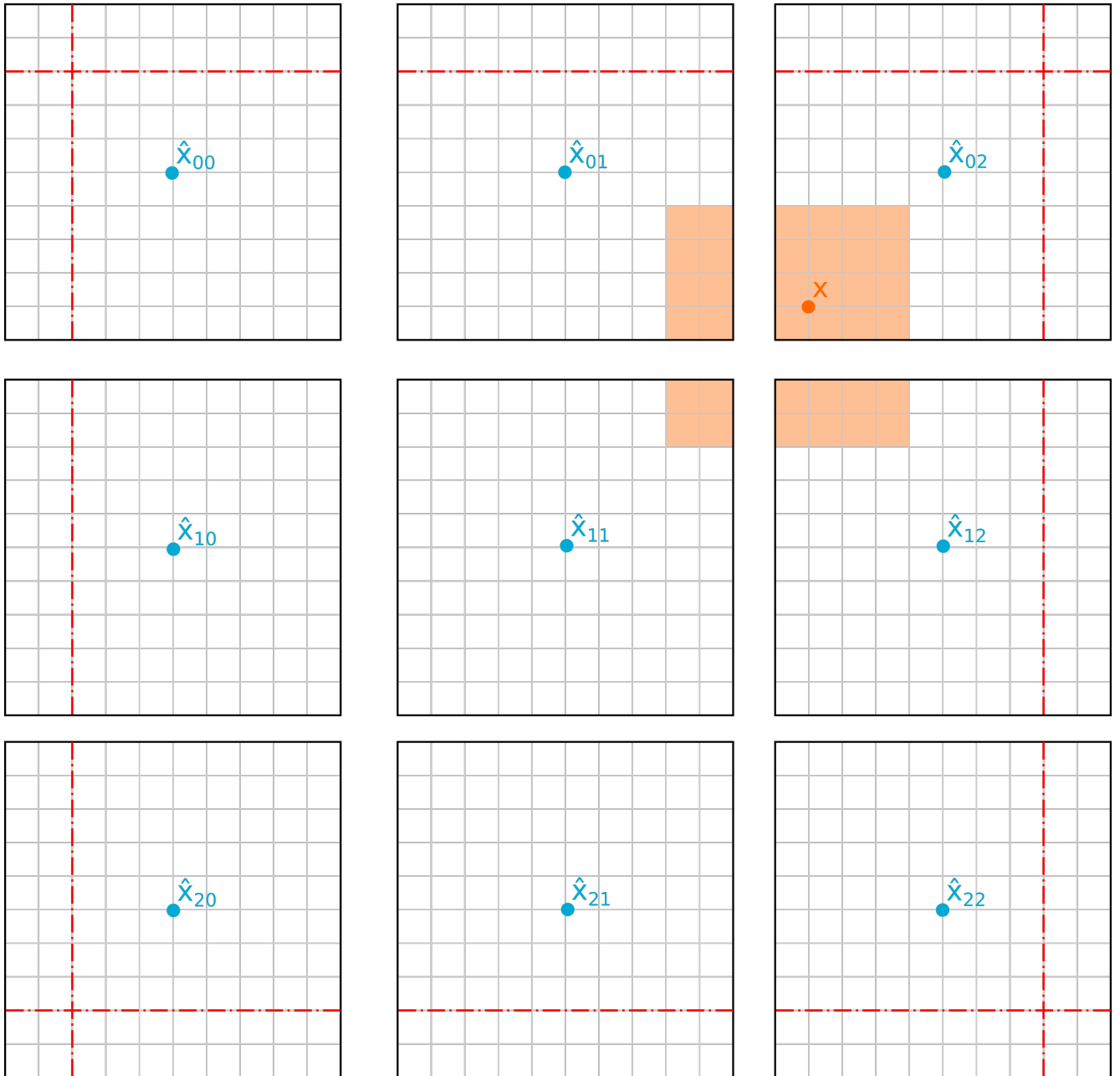


Рисунок 4.15 — Двумерная реализация локально карты проходимости пространства с неблокирующими операциями

4.6 Выводы

В данной статье было предложено три последовательных модификации локальной карты проходимости пространства и операций применяемых к ней карты и алгоритма ее построения?

, направленных на снижение вычислительной сложности производимых операций при условии наличия нескольких источников данных, работающих параллельно и независимо друг от друга. Первая увеличивала размер масси-

ва, хранящего карту, с целью снизить стоимость сдвига карты. Вторая меняла формат элементов массива, которая позволила бы а) одновременное обновление карты несколькими источниками и б) избежать излишней синхронизации между всеми клетками карты, тем самым снизив вычислительную сложность обновления. Последняя модификация разбивала карты на регионы и позволяла динамически подгружать новые, не прерывая операции обновления/получения карты, тем самым делая структуру данных полностью неблокирующей.

Были подготовлены реализации оригинальной версии и предложенных модификаций для одномерного случая. Для этих реализаций мы произвели измерения времени выполнения разных операций над картой, при разных её размерах. Исследуемые операции включали: сдвиг, получение карты, добавления одного фрагмента на карту, одновременное добавление нескольких фрагментов на карту, стандартную итерацию «публикации» карты – несколько сдвигов карты с добавлением фрагментов, после которых выполнялась операция получения.

Результаты экспериментов подтвердили теоретическую вычислительную сложность алгоритмов:

1. Для сдвига исходная реализация имеет вычислительную сложность $\Theta(N)$, все предлагаемые модификации имеют (амортизационную) сложность $\Theta(1)$. При этом экспериментально вычисленная константа наименьшая у последней модификации с полностью неблокирующими операциями – 103 нс, далее идёт карта с запасом – 1930 нс, и карта с неблокирующей операцией обновления – 4040 нс.
2. Операция обновления содержимого имеет линейную сложность для исходной реализации и карты с запасом. При этом сложность для карты с запасом больше ввиду большего количества обновляемых клеток. У карты с неблокирующей операцией обновления и карты с полностью неблокирующими операциями константная вычислительная сложность: 2510 и 3370 соответственно.
3. Для получения локальной карты проходимости пространства все алгоритмы имеют линейную сложность $\Theta(N)$, при этом у модификаций с неблокирующими операциями вычислительная сложность на несколько порядков выше: 23.3 для карты с неблокирующим обновлением, 22.2 для карты со всеми неблокирующими операциями, при 0.06 у исходной реализации и карты с запасом.

4. Для обновления карты несколькими источниками данных (одновременном, для тех реализаций где это возможно), результаты аналогичны эксперименту с единичным обновлением карты. При увеличении числа источников данных наблюдается уменьшение «оптимального» размера карты, при котором модификации с неблокирующими операциями становятся вычислительно эффективнее исходной реализации.
5. В эксперименте с симуляцией одной итерации публикации карты все алгоритмы показали линейную сложность $O(N)$. При этом исходная реализация имеет линейный рост с коэффициентом 2430 нс/пикс, карта с запасом – 377 нс/пикс (в 6.45 раз быстрее исходной реализации), карта с неблокирующей операцией обновления – 211 нс/пикс (в 11.52 раз быстрее исходной реализации), карта с полностью не блокирующими операциями 175 нс/пикс (в 13.86 раз быстрее исходной реализации)

Было предложено расширение для двумерного сценария карты с полностью неблокирующими операциями, показавшей лучшую производительность в последнем эксперименте. В дальнейших работах мы планируем подготовить реализацию данного расширения и провести его тестирования на реальных данных.

4.7 Исследование ограничений алгоритма построения карты проходимости пространства с блокирующими операциями

4.8 Модификации алгоритма построения карты проходимости пространства с блокирующими операциями

4.9 Исследование вычислительной сложности модификаций алгоритма построения карты проходимости пространства

4.10 Заключение к главе 4

Заключение

Основные результаты работы заключаются в следующем.

1. На основе анализа ...
2. Численные исследования показали, что ...
3. Математическое моделирование показало ...
4. Для выполнения поставленных задач был создан ...

И какая-нибудь заключающая фраза.

Последний параграф может включать благодарности. В заключение автор выражает благодарность и большую признательность научному руководителю Иванову И. И. за поддержку, помощь, обсуждение результатов и научное руководство. Также автор благодарит Сидорова А. А. и Петрова Б. Б. за помощь в работе с образцами, Рабиновича В. В. за предоставленные образцы и обсуждение результатов, Занудятину Г. Г. и авторов шаблона *Russian-Phd-LaTeX-Dissertation-Template* за помощь в оформлении диссертации. Автор также благодарит много разных людей и всех, кто сделал настоящую работу автора возможной.